

MEMORIA TÉCNICA DEL PROYECTO TFG LOOPSKILL

Pierangelo Guerrero / Alejandro Jiménez Ruiz
DAW II, IES LOPE DE VEGA

Contenido

1. Introducción.....	2
1.1 Objetivo	2
1.2 Descripción general del proyecto	2
1.3 Licencia del proyecto	3
2. Tecnologías empleadas	3
2.1 Angular standalone	3
2.2 TypeScript, HTML y SCSS.....	4
2.3 Servicios con LocalStorage para persistencia mock	4
2.4 Node.js y Express como arquitectura backend prevista	4
2.5 MySQL / MariaDB	5
2.6 GitHub, Figma y AWS.....	5
3. Desarrollo del proyecto: diseño y estructura	6
4. Desarrollo del proyecto: funcionalidades y etapas.....	14
5. Base de datos: diseño y estructura.....	16
6. Pruebas, errores cometidos y soluciones planteadas	17
7. Conclusiones.....	19
8. Bibliografía	20

1. Introducción

1.1 Objetivo

El objetivo de esta memoria técnica es documentar la primera versión funcional del frontend del proyecto LoopSkill, una plataforma web de formación online concebida como un entorno MOOC con recomendaciones personalizadas, control de acceso por planes y seguimiento del progreso del alumnado.

Esta versión del documento se centra en el trabajo desarrollado hasta el momento sobre el cliente web, con especial atención al diseño de las pantallas, la lógica funcional implementada, la estructura del código y las decisiones tomadas para que la aplicación pueda conectarse posteriormente a un backend real.

La memoria no pretende presentar el proyecto como un producto terminado, sino como una fase intermedia sólida y argumentada. Por eso, además de describir lo que ya está implementado, también se explican las limitaciones actuales, las decisiones que quedan aplazadas por depender del backend y las líneas de continuidad previstas para las siguientes iteraciones del TFG.

1.2 Descripción general del proyecto

LoopSkill nace con la idea de ofrecer una experiencia de aprendizaje online más clara, ligera y personalizada que la de muchas plataformas generalistas. En el anteproyecto se definió una aplicación de arquitectura cliente-servidor basada en Angular para el frontend, Node.js con Express para el backend y MySQL como sistema de base de datos, con despliegue previsto en AWS y un catálogo de cursos organizado por categorías, niveles y planes de suscripción.

En la primera entrega del TFG ya se habían planteado las pantallas principales del sistema —pantalla principal, detalle del curso, página de usuario, pantalla de login y sección de planes y suscripciones—, además de una primera base de datos poblada y coherente con el dominio del proyecto (Guerrero & Jiménez, 2026b). A partir de esa base, el desarrollo del frontend se ha orientado a construir una versión funcional en inglés, con una identidad visual propia y con una estrategia orientada al backend: la

lógica de negocio importante debe vivir en servicios y no quedar acoplada a mocks estáticos.

Durante esta fase se ha trabajado principalmente en la experiencia del usuario final: autenticación mock con persistencia local, onboarding de intereses, homepage dinámica, exploración del catálogo, detalle de curso, gestión de planes, seguimiento del aprendizaje, ajustes de cuenta y un primer panel de administración de cursos para perfiles con rol de administrador. A efectos de esta primera versión de la memoria, se propone una licencia MIT para el código del frontend, por ser permisiva, clara y adecuada para un proyecto académico con posible continuidad posterior.

1.3 Licencia del proyecto

El proyecto se distribuye bajo licencia MIT, una licencia de software libre y permisiva que permite reutilizar, modificar y distribuir el código manteniendo la referencia a la autoría original.

2. Tecnologías empleadas

2.1 Angular standalone

El frontend se ha construido con Angular utilizando el enfoque standalone. Esta decisión reduce la complejidad inicial de la aplicación, evita una dependencia excesiva de módulos y favorece una organización por funcionalidades. Cada pantalla principal del sistema —Home, Plans, Explore, Course Detail o Settings, entre otras— se implementa como un componente standalone, lo que mejora la legibilidad y hace más sencilla la evolución del proyecto.

Angular aporta además un sistema de routing claro, data binding bidireccional en formularios, directivas estructurales para el renderizado condicional y una buena separación entre vista, lógica y servicios. En el estado actual del proyecto ya se aprovechan estas capacidades para mostrar secciones según el estado del usuario, el plan contratado o la existencia de intereses seleccionados.

2.2 TypeScript, HTML y SCSS

TypeScript se ha utilizado como lenguaje principal de la lógica del frontend. Su sistema de tipado ayuda a mantener coherencia en modelos como User, Course o Plan, y reduce errores durante la evolución del código. En esta fase ha sido especialmente útil para reforzar la transición desde datos mock sencillos hacia servicios más estructurados.

HTML se emplea en las plantillas de cada componente, con un enfoque declarativo y legible. Por su parte, SCSS se ha usado para mantener una identidad visual consistente en toda la aplicación. Se ha definido un lenguaje visual limpio, moderno y minimalista, con el verde de LoopSkill como color principal, radios generosos, sombras suaves y bastante espacio entre bloques. Componentes como Course Detail, Plans, Settings o My Learnings comparten esa misma lógica visual, lo que refuerza la percepción de producto coherente.

2.3 Servicios con LocalStorage para persistencia mock

Aunque el backend aún no está integrado, el frontend no se ha construido como una simple maqueta estática. Para aproximar el comportamiento de una aplicación real se han creado servicios como AuthService, EnrollmentService y CourseService. Estos servicios encapsulan la lógica de acceso y actualización de datos, y utilizan LocalStorage como mecanismo de persistencia temporal.

Esta estrategia permite trabajar con un comportamiento cercano al real: los usuarios pueden registrarse, iniciar sesión, actualizar intereses, cambiar de plan, inscribirse en cursos o, en el caso de un administrador, gestionar el catálogo. La ventaja principal es que, cuando el backend esté disponible, la sustitución podrá hacerse dentro de los servicios sin tener que reescribir toda la interfaz.

2.4 Node.js y Express como arquitectura backend prevista

En la fase actual del proyecto, el backend ya no se plantea solo como una previsión arquitectónica, sino como una base técnica parcialmente implementada siguiendo la línea definida en el anteproyecto: Node.js con Express como API REST y MySQL como sistema de persistencia. En este estado ya se han desarrollado los elementos

iniciales de la capa servidor, como la configuración de la aplicación, la conexión con la base de datos y una primera funcionalidad de autenticación mediante registro, inicio de sesión y validación con JWT.

No obstante, el backend todavía no cubre toda la lógica funcional de la plataforma ni se encuentra plenamente integrado con el frontend. La mayor parte de las vistas del cliente continúan trabajando en modo mock, aunque han sido diseñadas a partir de entidades, relaciones y estructuras de datos coherentes con el modelo real ya definido en base de datos.

2.5 MySQL / MariaDB

La base de datos relacional del proyecto se ha modelado en MySQL/MariaDB. Ya existe un esquema con tablas para usuarios, planes, cursos, categorías, etiquetas, intereses del usuario, resultados de aprendizaje de cada curso e inscripciones. Además, durante esta fase se ha ampliado el diseño con la tabla `plan_features`, que permite representar de forma normalizada las ventajas de cada plan y usarlas tanto en backend como en frontend.

La estructura de la base de datos resulta especialmente relevante para el frontend porque condiciona el modelo de datos mostrado al usuario: planes Free, Pro y Premium; cursos asociados a un plan mínimo; intereses enlazados con etiquetas; y matrículas con porcentaje de progreso.

2.6 GitHub, Figma y AWS

GitHub se utiliza como repositorio principal del proyecto y como soporte para el trabajo en equipo. Esta herramienta permite versionar el frontend, compartir avances con el compañero responsable del backend y mantener trazabilidad sobre la evolución del sistema.

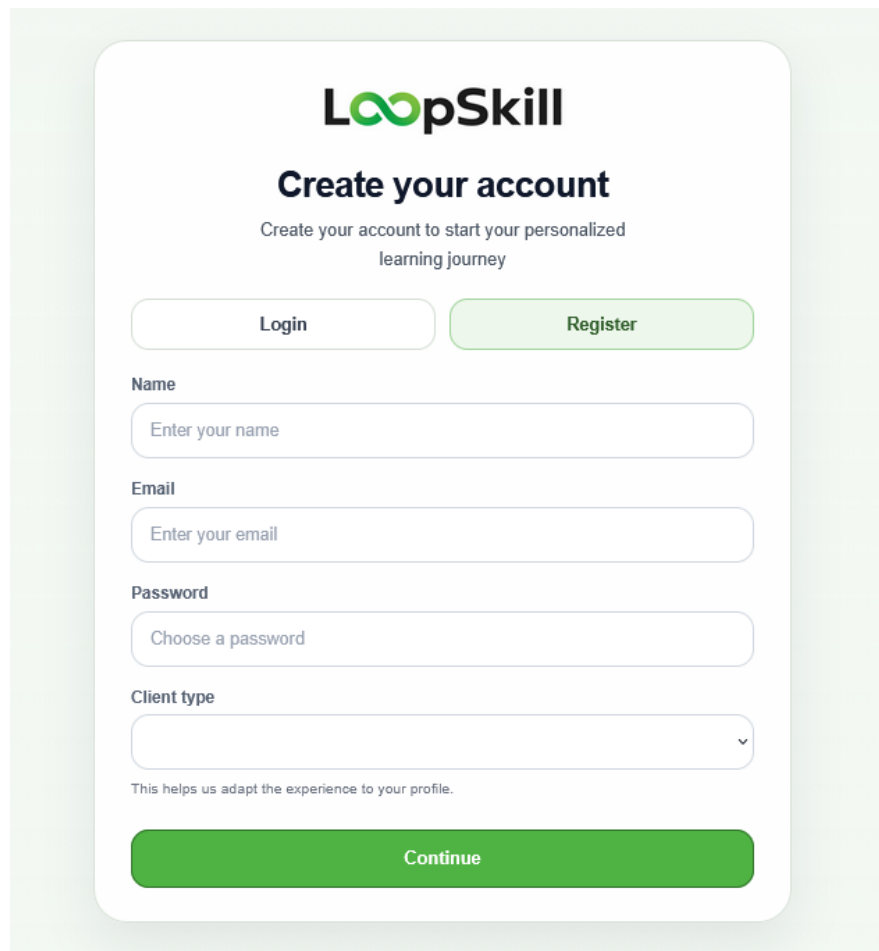
Figma se empleó en las fases iniciales para definir las pantallas principales y la experiencia visual general. AWS, por su parte, sigue siendo la plataforma prevista para el despliegue final, aunque en esta fase no se ha completado todavía una integración funcional de la aplicación en la nube. Estas herramientas ya forman parte del

ecosistema del proyecto, aunque su peso actual es distinto al de Angular o LocalStorage.

3. Desarrollo del proyecto: diseño y estructura

La estructura del frontend se ha organizado por funcionalidades, de forma que cada bloque importante del sistema tenga su propio componente y sus propios estilos. Este enfoque facilita el mantenimiento del proyecto y permite que las pantallas evolucionen sin generar una arquitectura rígida ni difícil de escalar.

A nivel visual se ha buscado una interfaz clara y de apariencia cuidada, sin llegar a ser recargada. El fondo general utiliza tonos muy claros, verdosos o grisáceos; las cards son blancas con bordes suaves; el verde de marca se reserva para las acciones principales; y los estados secundarios usan tonos más apagados para no competir con el CTA principal. Se ha tomado como referencia interna la coherencia entre Course Detail, My Learnings y Settings, y se ha reutilizado el símbolo de loop o infinito de la marca como elemento decorativo en algunos hero sections.



The image shows a web form for creating an account on LoopSkill. The form is centered on a light green background. At the top is the LoopSkill logo, followed by the heading 'Create your account' and a subtext 'Create your account to start your personalized learning journey'. Below this are two buttons: 'Login' and 'Register'. The 'Register' button is highlighted in green. The form then asks for 'Name', 'Email', and 'Password', each with a corresponding input field. Below these is a 'Client type' dropdown menu. A small note states 'This helps us adapt the experience to your profile.' At the bottom is a large green 'Continue' button.

LoopSkill

Create your account

Create your account to start your personalized learning journey

[Login](#) [Register](#)

Name

Email

Password

Client type

This helps us adapt the experience to your profile.

[Continue](#)

La pantalla de autenticación se separó del resto del layout para mantener el foco en el acceso. El flujo no termina en el registro, sino que continúa con una pantalla de onboarding de intereses, lo que permite construir desde el principio la capa de personalización. Este onboarding admite tanto la selección de intereses como la opción de omitirlos temporalmente.

Welcome back, **Alejandro Martin**.

Continue where you left off and keep building momentum.

Current course

Angular from Scratch

0% completed

Resume learning

Angular



0%

Current progress

Angular from Scratch

Progress: 0%

Most Popular Courses

Discover courses that match your interests.

Python



Python Fundamentals

This course introduces Python programming through practical examples and core programming concepts.

Free

View course

Angular



Angular from Scratch

Build modern frontend applications with Angular, components, routing and services.

Pro

View course

React



React Fundamentals

Create dynamic user interfaces with React using reusable components, hooks and state.

Premium

View course

SQL



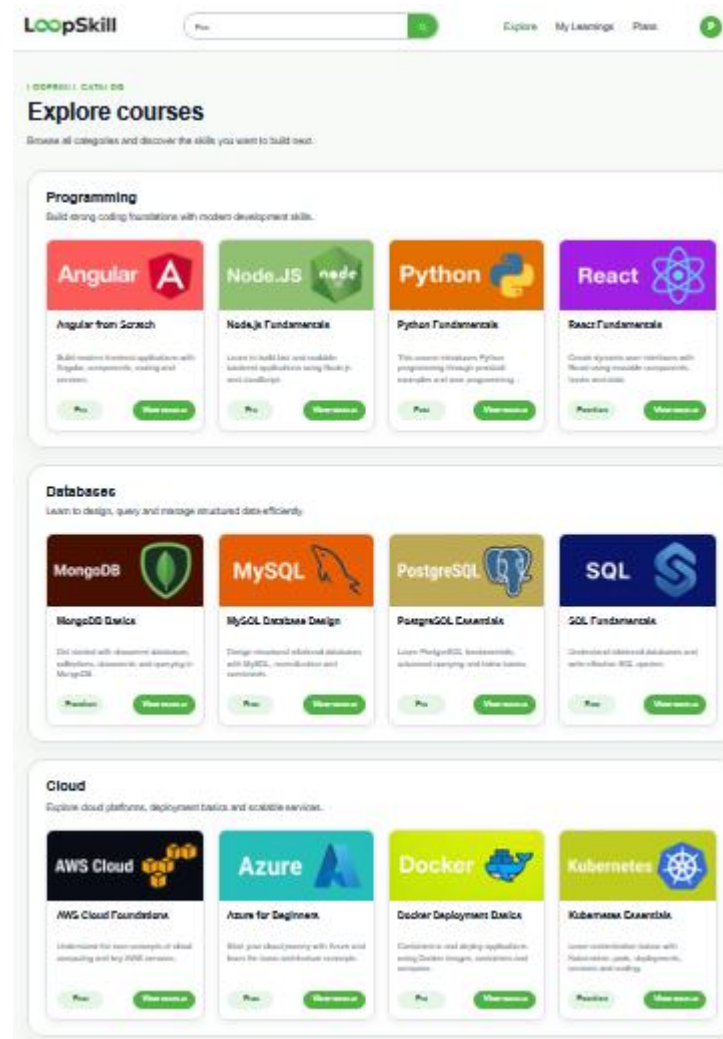
SQL Fundamentals

Understand relational databases and write effective SQL queries.

Free

View course

La página Home está organizada en secciones bien jerarquizadas: hero superior, cursos populares, recomendaciones personalizadas y categorías. El hero cambia según el estado del usuario: si todavía no ha empezado ningún curso, se muestra una variante orientada al descubrimiento; si ya tiene cursos inscritos, el enfoque cambia hacia continuar el aprendizaje.



Explore funciona como catálogo general. Muestra todas las categorías disponibles y agrupa los cursos dentro de cada una, ordenados alfabéticamente. Además, soporta navegación contextual desde la Home mediante query params, de manera que el usuario puede llegar a la categoría seleccionada con una transición natural.

PROGRAMMING

Python Fundamentals

This course introduces Python programming through practical examples and core programming concepts.

Beginner

Free

python

oop

Start course

Course overview

INSTRUCTOR

Laura Bennett

DURATION

18 hours

LESSONS

42 lessons

What you'll learn



Write basic Python programs using variables, loops and functions.



Understand core programming logic and syntax.



Solve beginner-friendly coding exercises.



Build confidence to continue into more advanced Python topics.

Course Detail es una de las vistas más completas del sistema. Incluye un hero amplio con metadatos del curso, etiquetas, CTA principal condicionado por el estado del usuario y bloques inferiores con overview, resultados de aprendizaje, habilidades y descripción extensa. En esta pantalla ya se aplica el control de acceso por plan y la lógica de inscripción.

LOOPSKILL DASHBOARD

My Learning

Track your current courses and revisit the ones you have already completed.

In Progress

Completed

**Node.js Fundamentals**

Programming · Intermediate

Progress: 72%

Started on: Mar 8, 2026

Resume

**Angular from Scratch**

Programming · Intermediate

Progress: 45%

Started on: Mar 8, 2026

Resume

**PostgreSQL Essentials**

Databases · Intermediate

Progress: 0%

Started on: Apr 13, 2026

Resume

La antigua sección de usuario evolucionó hacia My Learnings, con dos estados principales: cursos en progreso y cursos completados. Se incorporó también un empty state específico para la pestaña Completed, con un tratamiento visual coherente con el resto del producto. La gestión de la cuenta personal se desplazó a una nueva pantalla Settings, más apropiada para editar datos del usuario.

PLANS & PRICING

Choose the plan that supports your learning path

Compare available plans, review what is included in each option and upgrade your access whenever you are ready to unlock more courses.

YOUR CURRENT PLAN **Pro**

Free

Start learning with access to essential courses and core platform features.

✓ Access to beginner courses

✓ Personalized recommendations

✓ Progress tracking

✓ Learning dashboard

Not available

Most popular **Current plan**

Pro

\$12.99 / month

Unlock a wider catalog with intermediate content and a more complete learning experience.

✓ Everything in Free

✓ Access to intermediate courses

✓ Expanded course catalog

✓ Priority access to new content

✓ More advanced learning paths

Current plan

Premium

\$24.99 / month

Get full access to the complete catalog and premium learning features.

✓ Everything in Pro

✓ Access to advanced courses

✓ Full catalog access

✓ Premium learning experience

✓ Priority support

Upgrade to Premium

La pantalla Plans consolida la lógica de suscripción. Presenta los tres planes definidos en la base de datos, muestra el plan actual del usuario y permite simular upgrades mediante actualización del usuario en LocalStorage. En escritorio se mantiene una comparación amplia de características; en móvil se simplifica la visualización para no perjudicar la legibilidad.

USER SETTINGS

Manage your account

Update your personal information and choose the new skills you want to learn.

A

Alejandro Martin

alejandro.martin@loopskill.com

CLIENT TYPE

professional

CURRENT PLAN

Premium

Account

Password

Interests

Admin panel

Account

Update the main information associated with your LoopSkill account.

Name

Alejandro Martin

Email

alejandro.martin@loopskill.com

Client type

professional

Current plan

Premium

Manage plan

Save account changes

Por último, Settings concentra la gestión de cuenta, contraseña e intereses. Además, para los usuarios con rol de administrador, incorpora una pestaña adicional de administración del catálogo. Esta decisión responde a lo prometido en el anteproyecto respecto a la gestión de roles, con una implementación proporcionada al alcance actual del frontend.

4. Desarrollo del proyecto: funcionalidades y etapas

El desarrollo funcional del frontend se organizó por etapas, priorizando primero la navegación visible y la experiencia del usuario final, y después la robustez interna de los servicios.

La primera etapa consistió en levantar el proyecto Angular, definir el routing principal, introducir una identidad visual coherente y crear los componentes base. La segunda se centró en la autenticación mock: login, registro y persistencia del usuario en LocalStorage. En esta fase ya se decidió que el registro no incluyera la selección de intereses dentro del mismo formulario, sino que derivara a una pantalla específica de onboarding. Esta separación mejoró tanto la claridad del flujo como la futura mantenibilidad.

La tercera etapa abordó la experiencia central del producto: Home, Explore y Course Detail. Se construyó un catálogo orientado a tecnologías concretas, con cursos de programación, bases de datos, cloud, ciencia de datos y DevOps. También se añadió la lógica de recomendaciones por intereses y la posibilidad de inscribirse en cursos.

La cuarta etapa se orientó al seguimiento del progreso. Mediante EnrollmentService se resolvió la lectura de matrículas, la detección de cursos en progreso y la actualización del comportamiento del hero en Home. Esto permitió eliminar dependencias directas de mocks rígidos y acercar la interfaz a un estado más realista.

La quinta etapa se centró en Plans y Settings. En Plans se consolidó la gestión del plan actual del usuario y la lógica de upgrade mock. En Settings se incorporaron la edición de nombre y email, el cambio de contraseña, la gestión de intereses y la visualización del plan actual con acceso a la página de planes.

La etapa más reciente introdujo el rol de administrador. Sin añadir un campo artificial, se aprovechó el campo role ya presente en el modelo de usuario. Si el usuario tiene rol admin, Settings muestra una pestaña de panel de administración desde la que se puede visualizar el catálogo, crear cursos nuevos y eliminar cursos existentes. La edición de cursos queda identificada como la siguiente mejora funcional inmediata.

La tabla siguiente resume el alcance real de la versión actual:

Bloque funcional	Estado actual	Observaciones
Autenticación y registro	Implementado	Persistencia mock con AuthService y LocalStorage
Onboarding de intereses	Implementado	Condiciona recomendaciones en Home
Home, Explore y Course Detail	Implementado	Núcleo visible del producto ya operativo
Planes y suscripciones	Implementado en modo mock	Upgrade visual y control de acceso por plan
My Learnings y progreso	Implementado	Basado en EnrollmentService
Settings de usuario	Implementado	Cuenta, contraseña e intereses
Panel admin de cursos	Parcial	Create y Delete listos; Edit pendiente
Integración con backend	Pendiente	Bloqueada por falta de API disponible
Despliegue de proyecto en AWS	Pendiente	Último paso, tras conectar frontend y backend,

5. Base de datos: diseño y estructura

La base de datos del proyecto responde a una estructura relacional coherente con el dominio MOOC. Existen tablas principales para `categories`, `plans`, `tags`, `users`, `courses`, `course_outcomes`, `course_tags`, `user_interests` y `enrollments`. Cada una cumple una función concreta y permite modelar relaciones reales del sistema, como la asociación entre usuario e intereses, el vínculo entre curso y plan requerido o el progreso acumulado en una inscripción.

En `users` se almacenan los datos principales del usuario, incluyendo `role`, `client_type` y `plan_id`. Esto ha permitido que el frontend se apoye en campos ya existentes en el modelo para distinguir, por ejemplo, entre estudiante y administrador sin necesidad de introducir atributos artificiales. En `courses` se guardan título, descripción, categoría, nivel, plan requerido, imagen, duración, número de lecciones e instructor, lo que se corresponde bien con las necesidades actuales de Home, Explore, Course Detail y el panel de administración.

Las tablas `course_tags` y `user_interests` permiten establecer una lógica de recomendación basada en coincidencias de intereses. Aunque el algoritmo todavía es básico en esta fase del frontend, el modelo de datos ya soporta una evolución posterior hacia recomendaciones más elaboradas. Por su parte, `enrollments` representa la relación entre usuario y curso, e incorpora el porcentaje de progreso, que es la clave sobre la que se apoya My Learnings.

Durante esta fase se añadió una mejora al esquema mediante la tabla `plan_features`. Esta tabla relaciona cada plan con una lista ordenada de características, lo que evita almacenar listas embebidas poco escalables y permite representar de forma más correcta la comparación entre Free, Pro y Premium. Aunque en el frontend actual algunas características todavía se muestran desde datos mock, el modelo relacional ya queda preparado para que el backend sirva esta información de manera consistente.

Desde la perspectiva del frontend, la estructura de la base de datos no se ha usado aún mediante consultas HTTP reales, pero sí ha condicionado la forma de modelar los datos y la semántica de las pantallas. Esto es importante de cara a la memoria técnica

porque demuestra que el diseño del cliente no se ha improvisado, sino que se ha alineado con un esquema relacional previamente planificado.

6. Pruebas, errores cometidos y soluciones planteadas

A lo largo del desarrollo se han realizado pruebas manuales continuas, centradas tanto en el comportamiento funcional como en la consistencia visual entre componentes. Dado que la aplicación aún no consume un backend real, la mayor parte de las pruebas se han efectuado sobre rutas, formularios, persistencia local y estados de la interfaz.

Uno de los problemas detectados fue la dependencia excesiva de mocks estáticos para la lógica de negocio. Un ejemplo claro apareció en el comportamiento del hero de Home tras inscribirse en un curso: la detección del estado del usuario dependía de un mock rígido de matrículas en lugar de un servicio. Una vez corregido para usar `EnrollmentService`, un usuario recién inscrito dejó de ver la versión del hero pensada para quien aún no ha empezado ningún curso.

Otro error frecuente estuvo relacionado con datos persistidos en `LocalStorage` que no coincidían con los mocks actualizados. Esto ocurrió, por ejemplo, con el valor `clientType` de un usuario que seguía apareciendo en español porque había quedado almacenado en una sesión anterior. La solución consistió en limpiar las claves de `LocalStorage` y asegurarse de que el servicio inicializa datos mock solo si el almacenamiento aún no existe.

También se detectaron ajustes necesarios en la experiencia responsive, especialmente en la página `Plans`. La tabla comparativa completa resultaba poco viable en pantallas pequeñas, por lo que se optó por mantenerla en escritorio y sustituirla en móvil por una versión más resumida y vertical. Del mismo modo, se revisaron espacios, tamaños de cards y separación de CTAs para que el diseño no se degradara en dispositivos estrechos.

En `Settings` surgió la necesidad de diferenciar con más claridad la parte de cuenta personal de la parte de aprendizaje. Inicialmente, el avatar de la navbar redirigía a `My`

Learnings, pero esta decisión no era coherente con los patrones habituales de producto. La solución fue crear una pantalla Settings propia y reservar My Learnings exclusivamente para el seguimiento de cursos.

La tabla siguiente recoge un resumen de las incidencias más relevantes:

Incidencia	Causa principal	Solución adoptada
Hero no cambiaba tras inscribirse	Lectura desde mock de matrículas	Uso de EnrollmentService
Valor clientType incoherente	Dato antiguo persistido en LocalStorage	Limpieza de claves y reinicialización
Plans poco usable en móvil	Tabla comparativa demasiado ancha	Ocultar comparación en móvil y simplificar layout
Avatar llevaba a My Learnings	Navegación poco natural	Creación de Settings como destino de cuenta
Gestión admin no cubierta	Falta de interfaz por rol	Pestaña Admin panel visible solo para administradores

En conjunto, estas incidencias han servido para reforzar dos ideas: la primera, que incluso en una fase mock conviene construir la lógica en servicios y no en datos importados de forma rígida; y la segunda, que una buena parte de la calidad percibida del frontend depende tanto de la funcionalidad como del ajuste fino del layout y la navegación.

7. Conclusiones

Esta primera versión del frontend de LoopSkill puede considerarse una base sólida y defendible. A nivel visual existe ya una identidad clara y coherente. A nivel funcional, el usuario puede registrarse, iniciar sesión, seleccionar intereses, explorar el catálogo, ver el detalle de los cursos, inscribirse, consultar su progreso, gestionar su cuenta y cambiar de plan en modo mock. Además, se ha incorporado una primera aproximación a la gestión de roles mediante un panel de administración visible únicamente para usuarios con rol de administrador.

El principal límite de esta fase es la ausencia de integración con un backend real. No obstante, este límite no invalida el trabajo realizado, ya que buena parte del frontend se ha construido precisamente para facilitar esa integración posterior. La existencia de servicios propios, la persistencia local controlada, la alineación con el modelo de datos y la separación por funcionalidades son decisiones que reducen de forma notable el coste de la siguiente etapa.

Como trabajo inmediato para la siguiente versión, resultan prioritarios tres frentes: completar la edición de cursos dentro del panel de administración, sustituir la lectura directa de `MOCK_COURSES` por `CourseService` en todas las pantallas del catálogo, y conectar finalmente la aplicación con la API del backend cuando esta esté disponible. A partir de ahí, LoopSkill podrá evolucionar desde una versión funcional de frontend hacia un sistema integrado y plenamente desplegable.

En definitiva, el estado actual del proyecto cubre una parte muy significativa de lo prometido en el anteproyecto y en la primera entrega. El frontend ya no es una simple demostración visual, sino una aplicación consistente, razonada y preparada para crecer de forma ordenada.

8. Bibliografía

- Amazon Web Services. (s. f.). AWS documentation. aws.amazon.com/documentation
- Angular. (s. f.). Angular documentation. angular.io
- Express.js. (s. f.). Express documentation. expressjs.com
- MySQL. (s. f.). MySQL documentation. dev.mysql.com
- Node.js. (s. f.). Node.js documentation. nodejs.org